Designed by Remus Teodorescu 2026

LEARNING OBJECTIVES

What You Will Be Able To Do

THE GOALS, IN PLAIN WORDS

- **explain** — why a dense layer on an image is the wrong object, in two ways
- **state** — what a kernel computes, and count the parameters in a layer
- **distinguish** — equivariance from invariance, and say which your task needs
- **write** — the message-passing update, and say why the sum matters
- **test** — a model for permutation invariance, and say what failing it means
- **choose** — between a grid, a graph and neither

WHAT TODAY ASSUMES

from L4.2	depth, capacity, the vanishing gradient
from L4.1	what a layer computes
not needed	any image processing background

ONE PRINCIPLE, TWICE

A learned local rule, applied everywhere. Convolution and message passing are that idea on two different neighbourhood structures.

AND ONE OBJECT RETURNS

The six-bus network today, again in L12.1, and built by you in Ex_12.1.

Why a Fully Connected Layer on an Image Is Absurd

THE QUESTION I LEFT YOU WITH

- **a 256×256 greyscale image** — is 65,536 numbers
- **connect that to a thousand hidden units** — 65 million parameters, in the first layer alone
- **but the count is the smaller problem** — the layer has no idea neighbouring pixels are related
- **it must discover from data** — what you already know for certain about the world

THE COUNT

256 × 256 image	65,536 inputs
→ 1,000 units, dense	65.5 million weights
→ 1,000 units, 3×3 conv	about 9,000
ratio	roughly 7,000 to 1

THE ARGUMENT THAT MATTERS MORE

If you already know something about the structure of your input, building it into the architecture is free. Making the network learn it from data costs you parameters, data and time — and it may not learn it at all.

A Learned Local Rule, Applied Everywhere

THE ONE SENTENCE THIS HALF-LECTURE IS BUILT ON

Take a small function of a neighbourhood. Learn its parameters. Apply the same function at every location. Everything else today is a detail of what "neighbourhood" means.

ON A REGULAR GRID — CONVOLUTION

The neighbourhood is a small square of pixels. The rule is a weighted sum with a shared set of weights, called a kernel.

Slide the kernel over the image and you get a new image. Nine parameters, applied sixty-five thousand times.

Because the same kernel is used everywhere, a feature detected in the corner is detected in the centre with the same weights. You did not have to teach it twice.

ON A GRAPH — MESSAGE PASSING

The neighbourhood is whatever the edges say it is. There is no notion of up, down or adjacent-by-one-pixel; there is only connected.

The rule gathers information from the connected nodes, combines it with the node's own state, and updates.

Because the same rule is used at every node, a six-bus network and a six-hundred-bus network use the same parameters. Renumber the buses and nothing changes.

Weight sharing over positions, or weight sharing over nodes. One principle, and it is the reason both architectures work.

The Kernel, and What It Does

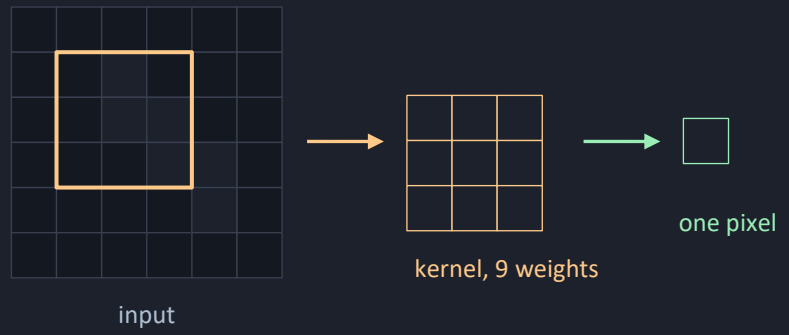
A WEIGHTED SUM OVER A SMALL WINDOW

- **a 3×3 window** — multiply, sum, add a bias, apply the activation
- **move one pixel and repeat** — with the same nine weights
- **the nine weights are learned** — nobody designs them
- **before 2012** — an engineer hand-designed edge and corner detectors

$$h_{ij} = a\left(\sum_m \sum_n k_{mn}\, x_{i+m,\, j+n} + b\right)$$

Two-dimensional discrete convolution — the whole first half of this lecture.

ONE WINDOW, ONE OUTPUT PIXEL



Slide the amber window over every position and you have built the whole output image — with the same nine numbers each time. That reuse is the entire saving, and it is also why the detector works anywhere in the frame.

Stride, Padding and Channels

THREE KNOBS, AND ONE OF THEM MATTERS MOST

- **stride** — how far the window moves. Two halves the resolution.
- **padding** — what happens at the edge. Zeros keep the size.
- **channels** — the one students find hardest
- **the kernel is 3×3×input-channels** — one of those per output channel
- **so count carefully** — this is where parameter estimates go wrong

$$P_{\text{dense}} = N_{\text{in}} N_{\text{out}} \qquad P_{\text{conv}} = K^2 C_{\text{in}} C_{\text{out}}$$

THE THREE KNOBS

kernel size	3×3 almost always
stride	1 to keep size, 2 to halve it
padding	'same' keeps the size
channels	how many features per position

THE COUNT THAT SURPRISES PEOPLE

A layer from 64 channels to 64 channels with 3×3 kernels holds 64 × 64 × 9, about 37,000 weights — and it is completely independent of the image size. The same layer works on a thumbnail and on a gigapixel scan.

Pooling, and Why Resolution Falls

TRADING WHERE FOR WHAT

- **pooling** — replace each block with one number, usually its maximum
- **it reduces computation** — obviously
- **and makes the representation** — slightly insensitive to small shifts
- **so resolution falls as channels rise** — position traded for meaning
- **right for classification** — wrong for segmentation, which needs both

RESOLUTION DOWN, CHANNELS UP

input	256 × 256 × 1
conv + pool	128 × 128 × 32
conv + pool	64 × 64 × 64
conv + pool	32 × 32 × 128
flatten + dense	3 classes

A typical classification network, in five lines. Notice that the total number of values per layer stays roughly constant — you are trading spatial detail for feature detail, not simply throwing information away.

THE PROPERTY

Equivariance, and Why It Is Not Invariance

TWO WORDS THAT GET USED INTERCHANGEABLY AND SHOULD NOT

- **equivariant** — move the input, the output moves with it
- **invariant** — move the input, the answer does not change
- **convolution gives equivariance free** — the same kernel applied everywhere
- **invariance is built up by pooling** — deliberately discarding position
- **which you need** — depends on whether your output should move

WHAT EACH BUYS

equivariant	output moves with the input
invariant	output unchanged by the move
convolution	equivariant, by construction
pooling	a step towards invariance
global average	invariant — position discarded
segmentation	wants equivariance throughout

And here is the version that matters in Part 2. A graph network is equivariant to relabelling the nodes. That is the same property, on a different symmetry, and it is the whole argument for using one on a power network.

What the Layers Actually Learn

A HIERARCHY NOBODY DESIGNED

- **first layer** — oriented edges and colour blobs
- **second and third** — corners, junctions, textures
- **deeper** — object parts, then whole objects
- **nobody put that hierarchy in** — it emerges from the architecture and the loss
- **and it is why transfer learning works** — early layers are not task-specific

THE HIERARCHY

layer 1	oriented edges, blobs
layer 2 – 3	corners, junctions, texture
middle layers	parts — a wheel, a weld toe
late layers	whole objects, whole defects
output	the label you asked for

This is why a network trained on a million photographs of everyday objects is a good starting point for a weld inspection task with four hundred images. You keep the early layers and retrain the late ones — which is transfer learning, and L6.2 covers it properly.

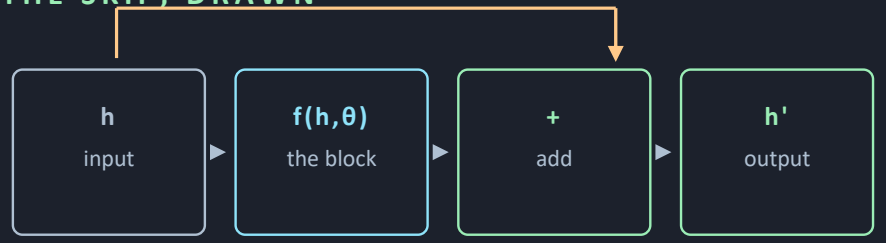
Residual Connections

THE FIX FOR THE PROBLEM IN L4.2 SLIDE 8

- **L4.2 slide 9** — gradients vanish along a long product
- **the residual connection** — learn the change, not the next representation
- **a direct path to every layer** — passing through no weights at all
- **the other reading** — each block only learns what to correct
- **a block with nothing to add** — can output zero, which is easy to learn

$$\mathbf{h}_{k+1} = \mathbf{h}_k + f(\mathbf{h}_k, \theta_k)$$

THE SKIP, DRAWN



the skip — no weights on this path

Networks went from about twenty trainable layers to over a hundred within a year of this idea, and it is now in essentially every deep architecture — including the transformer you meet after the break.

IN THIS COURSE

You will not need a hundred layers. But you will meet skip connections in L11's camera network, and now you know why they are there.

Normalisation, Briefly

KEEPING THE NUMBERS IN RANGE, LAYER BY LAYER

- **batch normalisation** — standardise activations, then rescale with learned parameters
- **the original explanation** — has not survived scrutiny
- **the honest account** — it smooths the loss surface and permits larger steps
- **it behaves differently in training and inference** — which catches people
- **rarely needed at our sizes** — included because you will read about it

THE FAMILY

batch norm	across the batch — vision
layer norm	across features — transformers
group norm	when the batch is tiny
none	usual in this course

THE BUG THIS CAUSES

model.eval() before inference and model.train() before training. Omit the first and your validation numbers are computed with batch statistics from whatever happened to be in the batch — which is both wrong and unstable.

WHERE YOU WILL USE IT

The Camera on the JetRacer

A CONCRETE NETWORK, ON HARDWARE YOU WILL DRIVE

- **L11** — a camera frame in, a steering angle out
- **regression, not classification** — no softmax, no global pooling
- **it runs on a Jetson Nano** — so the network must be small enough to be quick
- **and it is trained by imitation** — you drive, it records, the network copies

L11.1, IN OUTLINE

input	camera frame, downscaled
conv stack	a few layers, few channels
flatten	
dense	one output
output	steering angle

Small by any modern standard, and it has to be: the whole thing must run on the car, in real time, on battery. L6.2's deployment material is what makes that possible, and this is the network it is talking about.

THE TURN

Now Take Away the Grid

THE SAME IDEA, ON A DIFFERENT KIND OF NEIGHBOURHOOD

- **convolution works** — because an image has a fixed, regular neighbourhood
- **a power network does not** — bus 4 has two neighbours, bus 2 has three
- **and no natural left or right** — the grid structure is gone
- **pipe networks, molecules, FE meshes, supply chains** — all the same shape
- **so keep the principle** — and replace 'my eight neighbours' with 'whoever the edges connect'

IRREGULAR NEIGHBOURHOODS

Bus 2 has three neighbours. Bus 6 has two. No kernel can be written for that — but an update rule that sums over whatever neighbours exist can.

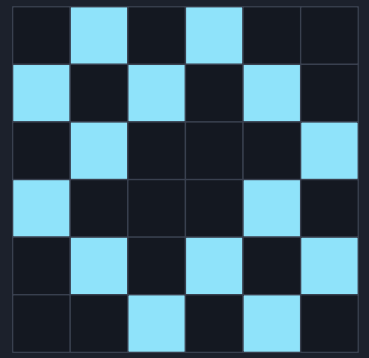
Nodes, Edges, and Two Matrices

EVERYTHING A GRAPH NETWORK CONSUMES

- **a graph is nodes and edges** — and two matrices write it down
- **the node feature matrix** — one row per node, holding what you measured there
- **the adjacency matrix** — a one wherever an edge exists
- **it is sparse** — and that sparsity is why graph networks scale

ADJACENCY, FOR THE SIX BUSES

A



Symmetric, because a line carries power both ways. Fourteen non-zero entries out of thirty-six — and a real transmission network is far sparser than this. The diagonal is empty because a bus is not connected to itself, though some formulations add it back deliberately, which is the next slide but one.

Message Passing — the Update Rule

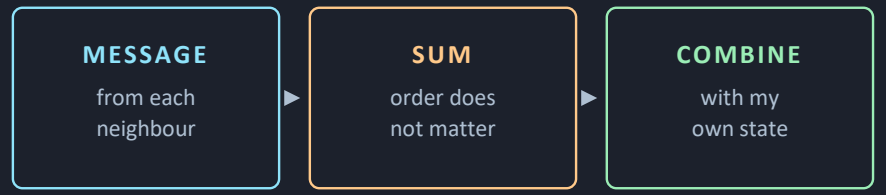
THREE STEPS, REPEATED AT EVERY NODE

- **for each node** — compute a message from each neighbour
- **add the messages up** — then combine with the node's own state
- **both functions are small networks** — and they are shared across every node
- **the sum matters more than it looks** — order-independent, and count-independent
- **which is what lets one model** — handle graphs of different sizes

The general form. Every graph network you will read about is a choice of φ and ψ.

$$\mathbf{h}_i^{(l+1)} = \phi\left(\mathbf{h}_i^{(l)}, \sum_{j \in \mathcal{N}(i)} \psi\left(\mathbf{h}_i^{(l)}, \mathbf{h}_j^{(l)}\right)\right)$$

ONE NODE'S UPDATE



Repeat for every node, with the same two functions. Then repeat the whole layer to reach two hops away.

HOW FAR INFORMATION TRAVELS

One layer reaches your neighbours. Two layers reach their neighbours. So the number of layers should be roughly the diameter of your graph — for six buses, about three. More than that and you are not gaining reach, you are just adding parameters.

Graph Convolution, in One Matrix Line

THE SIMPLEST USEFUL CHOICE OF THOSE TWO FUNCTIONS

- take the message to be the neighbour's state, unchanged
- **and the combine step** — a shared linear map, then an activation
- **read it right to left** — mix the features, sum over neighbours, apply the nonlinearity
- **add the identity** — so a node's own state is included
- **and normalise by degree** — the difference between training and exploding

$$\mathbf{H}^{(l+1)} = \sigma\left(\widetilde{\mathbf{D}}^{-1/2}\widetilde{\mathbf{A}}\,\widetilde{\mathbf{D}}^{-1/2}\mathbf{H}^{(l)}\mathbf{W}^{(l)}\right)$$

READING IT TERM BY TERM

H	node features, one row per node
W	shared weights — the learned part
Ã = A + I	neighbours, plus yourself
D̃	degree — how many neighbours
D̃^(−½) Ã D̃^(−½)	the normalised sum
σ	the activation

Compare this with a convolution. Same shape: gather a neighbourhood, apply shared weights, activate. The adjacency matrix is doing the job the sliding window did — it says which values belong to whose neighbourhood.

Renumber the Buses, Nothing Changes

THE ARGUMENT FOR THE WHOLE ARCHITECTURE

- **there is no natural order** — numbering the buses is bookkeeping
- **so a model whose answer changes when you renumber** — is wrong
- **not inaccurate** — wrong, in a way accuracy does not capture
- **a graph network cannot make that mistake** — permute the input, the output permutes
- **exactly, not approximately** — and Ex_05 measures how badly a dense net fails this

$$f(\mathbf{PX},\ \mathbf{PAP}^{\top}) = \mathbf{P}\, f(\mathbf{X}, \mathbf{A})$$

MEASURED IN EX_05 NOTEBOOK 03

graph network	1.7×10^{-16} rad
dense network	5.7×10^{-1} rad
after relabelling, GNN	accuracy unchanged
after relabelling, dense	153× worse

Those are the numbers from your own exercise. The graph network's gap is floating-point noise — it is not approximately invariant, it is invariant. The dense network's answer is essentially destroyed by an operation that changed nothing physical whatsoever.

Three Kinds of Question You Can Ask

NODE LEVEL — one answer per node

What is the voltage at every bus? What is the stress at every mesh point? The output has the same shape as the input, and this is what L12 does.

EDGE LEVEL — one answer per edge

What is the power flowing on every line? Is this connection about to fail? Less common, and it is what you want for line ratings.

GRAPH LEVEL — one answer per graph

Is this molecule toxic? Is this network stable? You aggregate over all nodes at the end, which is the graph analogue of global pooling.

INDUCTIVE OR TRANSDUCTIVE

A transductive model is fitted to one fixed graph and answers questions about that graph. It cannot be applied to a different one.

An inductive model learns a rule and can be applied to a graph it has never seen — a different topology, a different number of nodes.

Because the weights are shared across nodes rather than tied to particular ones, a graph network can be inductive. Whether yours is depends on how you train it.

This is the distinction L12.1 turns on, and it is why that lecture argues its own architecture is not obviously justified. Hold it until slide 20.

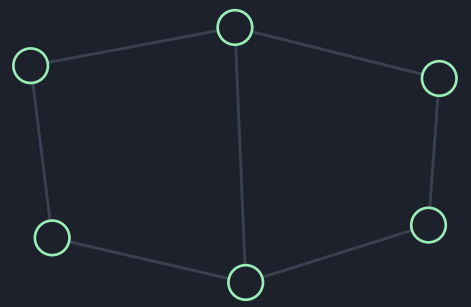
WHERE YOU WILL USE IT

The Six Buses, Three Times

THE SAME OBJECT, DELIBERATELY

- **Ex_05 notebook 03** — node regression on a six-bus network
- **L12.1** — the same six buses, in a physics-informed state estimator
- **Ex_12.1** — where you build that estimator
- **three encounters with one object** — and by the third it is familiar
- **which is why the exercise uses a real Newton–Raphson solve** — not a toy

THE SAME NETWORK, THREE TIMES



this week	Ex_05 nb03 — node regression
week 11	L12.1 — state estimation
week 11	Ex_12.1 — you build it

By the third time you will not be learning the network. You will be learning the physics.

JUDGEMENT

When a Graph Network Is Not the Answer

AN UNCOMFORTABLE RESULT FROM YOUR OWN EXERCISE

On the six-bus regression, the dense baseline is more accurate than the graph network — 0.0016 rad against 0.0043 — and a fifty-year-old DC linearisation beats them both at 0.0023. I did not arrange that. It is what the measurement says.

WHY THE GNN LOSES HERE

Six nodes is a tiny graph. A dense network can simply memorise the relationships, and with enough data it will.

The topology is fixed and known, so the structural information the graph network encodes approximately is available exactly by other means.

And in L12 the physics residual already contains the topology, in the admittance matrix — perfectly, not approximately. A graph network there would be learning a soft version of something you were handed for free.

That is precisely the argument L12.1 makes against its own architecture, and it is worth more than a slide claiming a win.

WHEN IT DOES EARN ITS PLACE

When the topology changes. The exercise's line-trip test shows the graph network degrading half as badly as the dense one — still not usable, but the ordering is clear and the reason is structural.

When you want one model for many networks. Weight sharing means a model trained on hundreds of configurations can face one it has never seen. A dense model cannot.

When the graph is large. Cost grows with edges rather than with nodes squared.

And when relabelling must not change the answer — which, on real operational data assembled from several sources, it must not.

Name what the architecture buys you in your formulation, not in general. That is the transferable habit.

Failure Modes

ZERO MEANS TWO DIFFERENT THINGS

Most nodes have no measurement. Feed zeros and the network learns that zero is a reading. Feed a mask channel alongside the value and it learns the value is absent. This single design choice is most of the problem in L12.

TOO MANY LAYERS — OVER-SMOOTHING

Each layer averages a node with its neighbours. Stack enough and every node converges to the same representation, and the network can no longer tell them apart. Match the depth to the graph diameter.

THE IMAGE IS THE WRONG SIZE

A convolutional stack assumes a fixed input size once you flatten into a dense layer. Change the camera resolution and it breaks — usually loudly, occasionally not.

WHAT TO CHECK, IN ORDER OF COST

free	print every tensor shape once
free	check the adjacency is symmetric
cheap	permute the nodes — does it hold?
cheap	shrink the depth to the diameter
real work	add a mask channel and retrain
real work	test on a changed topology

The permutation test on line three costs four lines of code and tells you immediately whether your model has the property you built it for. Run it before you report anything.

YOUR TURN

Ex_05 — Grids, Then Graphs

FIVE NOTEBOOKS, AND ONE OF THEM MATTERS MOST

- **notebook 01** — a small CNN on generated inspection images: crack, pit, clean
- **notebook 02** — message passing by hand in NumPy, before any library
- **notebook 03** — node regression on the six buses, with the permutation test
- **notebook 04** — a short sequence task, and the bridge into L5.2

THE NOTEBOOKS

00	environment check — read only
01	a CNN on inspection images
02	message passing, by hand
03	the six-bus network
04	sequences, and a look at attention
05	the report

THE QUESTION THAT CARRIES THE MARKS

The dense model was more accurate. Would you still use the graph network, and on what evidence?

SUMMARY

Key Takeaways

1	**One idea, two domains**	A learned local rule applied everywhere. Convolution on a grid, message passing on a graph. The rest is bookkeeping.
2	**Build in what you know**	Structure you put into the architecture is free. Structure the network must learn from data costs parameters, data and time.
3	**Equivariance is not invariance**	Convolution is equivariant to translation; a classifier should be invariant. A graph network is equivariant to relabelling — exactly.
4	**Depth means reach**	One graph layer reaches your neighbours. Match the layer count to the graph diameter; more is over-smoothing, not more reach.
5	**Fashion is not selection**	Your own exercise has the dense baseline beating the graph network on accuracy. Say what the architecture buys you in your formulation.

Next — L5.2: sequences, and the architecture that replaced recurrence.

OVER TO YOU

Questions

TEN TO TEST WHETHER TODAY LANDED

- **a dense layer on a 256-square image** — how many parameters, and what is the deeper objection?
- **what does a kernel compute** — and how many weights does it hold?
- **equivariance or invariance** — which does a segmentation model need?
- **why does resolution fall as channels rise** — and what is traded away?
- **why does a residual connection help** — as gradients, not intuition?
- why can convolution not be applied to a power network?

AND FOUR MORE

seven	write the message-passing update in words
eight	why sum the messages rather than concatenate?
nine	what does permutation invariance guarantee, exactly?
ten	when is a graph network the wrong choice?

THE ONE WITH NO SETTLED ANSWER

Your dense model passes the permutation test on your data. Does that make it invariant?

BEFORE L5.2

Run notebooks 01 and 02. Bring the permutation result.

SOURCES

References and Further Reading

THE BOOK

Prince, Understanding Deep Learning — chapter 10 for convolutional networks, chapter 11.2 and 11.4 for residual connections and normalisation, chapter 13 for graph neural networks. Chapter 13 is the only textbook treatment on this course's reading list, and it is a good one.

Goodfellow, Bengio & Courville — chapter 9 for convolution, and it is worth reading for the signal-processing framing Prince does not give. But note two traps: its chapter 10 is "Recurrent and Recursive Nets", where recursive means tree-structured composition and not message passing; and its chapter 16 is "Structured Probabilistic Models Using Graph Theory", which is Bayesian networks. Neither is a graph neural network. Goodfellow predates them entirely.

THE PRIMARY SOURCES

LeCun et al. (1998), Gradient-based learning applied to document recognition — LeNet, and still readable.

He et al. (2016), Deep residual learning for image recognition — the ResNet paper.

Kipf & Welling (2017), Semi-supervised classification with graph convolutional networks — the source of the matrix expression on slide 15.

Lam et al. (2023), Science 382 — GraphCast, which is this lecture at planetary scale.

READ IN THIS ORDER

before Ex_05 nb01	UDL ch. 10.1 – 10.3
before Ex_05 nb03	UDL ch. 13.1 – 13.4
if slide 15 was fast	Kipf & Welling — it is short
for the motivation	the GraphCast paper
before L5.2	UDL ch. 12.1 – 12.3

you will see these six again

NEXT

L5.2 — Sequences and Attention

Today's structures were spatial — a grid, a graph. Next time the structure is temporal, and the question becomes how to get information from the start of a long sequence to the end of it. Two answers, thirty years apart.

BEFORE THE NEXT SESSION

read	UDL ch. 12.1 – 12.3
run	Ex_05 notebooks 00 to 03
bring	your permutation-test numbers
think about	why a graph layer count should match the diameter

Bring the permutation numbers. They are the clearest evidence in the whole of Part 1 that an architectural choice can buy you something a bigger model cannot, and L5.2 opens by comparing them with the room's.